Security Risk Management Policy

Company	Rushing Technologies
Product	ZenFinance (iOS application and API)
Policy owner	Nicholas Rushing, Founder
Security contact	support@rushingtechnologies.com (monitored)
Version	1.0
Adopted	2026-07-14
Parent document	Information Security Policy §5

1. Purpose and Scope

This policy defines how Rushing Technologies identifies, assesses, treats, and monitors security risks to ZenFinance's systems and to the user data it handles. It elaborates §5 (Risk Management) of the Information Security Policy and inherits that policy's definitions, data classifications, roles, and exception process. It applies to all company systems, code, infrastructure, and third-party processor relationships.

2. Roles

The **Founder** is the risk owner: accountable for running assessments on the cadence below, deciding treatment, and recording outcomes. Rushing Technologies is a single-member company; the compensating controls for the absence of independent internal review are the automated gates in §4 and the external assurance path in §8.

3. Risk Identification

Risks are identified from these standing sources:

- **Automated scanning** of every code change and of dependencies (§4).
- **Runtime telemetry** — Sentry error and crash monitoring across the API and the iOS app, tagged by route and release.
- **Processor notices** — security advisories and incident notifications from Plaid, RevenueCat, Anthropic, Sentry, Railway, Cloudflare, GitHub, Apple, and Resend.

- **Architecture review triggers** — any new data category, third-party processor, or externally reachable surface (a mandatory trigger under the parent policy).
- **Failure-mode analysis** — the documented drill catalogue (`FAILURE_DRILLS.md`) covering provider webhook outage, item reauthentication, LLM provider failure, billing webhook failure, and account-deletion failure.
- **User and external reports** to the monitored security contact.

4. Risk Assessment Process and Cadence

Cadence	Activity
Every pull request	CI runs typechecking, the full API test suite, and a heuristic security scan; findings are reviewed before merge.
Release branches	An LLM-triaged security scan; CONFIRMED findings at or above the configured severity threshold fail the build (`.github/scripts/gate_security_scan.py`).
Monthly	Dependency vulnerability review: `npm audit --audit-level=high` gate; moderate advisories in non-production tooling are tracked and re-checked.
Per release	The release checklist: typecheck, API test suite against a dedicated test database, production build, dependency audit, native dependency verification.
Semi-annual	Full review of the Information Security Policy and this policy, including the risk register.
Event-driven	Architecture change triggers, processor security notices, and incidents each initiate an immediate assessment of the affected surface.

Assessment method. Each identified risk is evaluated for **likelihood** (exploitability given current controls and exposure) and **impact** (using the parent policy's data classifications — risks touching Restricted data, authentication, or money movement rank highest; Confidential data next; Internal/Public lowest). The combination determines severity, which maps to the remediation timelines in §6.

5. Risk Treatment

Each assessed risk receives one of four treatments, decided and recorded by the risk owner:

- **Remediate** — fix through the standard change pipeline (pull request, CI gates, review). This is the default for anything production-reachable.
- **Mitigate** — apply a compensating control (e.g., defense-in-depth dependency overrides for risky transitive paths, rate limiting on abuse surfaces) and track the residual risk.
- **Accept** — permitted only for risks not reachable in the production request path (e.g., moderate advisories confined to development tooling); acceptance is recorded with rationale and a re-check date.
- **Avoid** — remove the risky surface, dependency, or data category entirely; minimization is the preferred treatment for data-related risk (read-only bank scope, no card storage, redacted LLM inputs).

6. Remediation Timelines

Severity	Target
Critical / actively exploited	Remediated or mitigated before the next deploy, and within 72 hours of confirmation
High	Within 14 days
Moderate	Within 90 days, or documented acceptance where the affected path is not production-reachable

7. Risk Register and Records

`SECURITY_AUDIT.md` in the product repository is the risk register and remediation log: assessment results, applied remediations, accepted risks with re-check dates, and incident records are maintained there under version control, giving a complete, timestamped history. Failure-mode expectations and their validating automated tests are maintained in `FAILURE_DRILLS.md`.

8. Assurance and Improvement

Internal assessments are performed and documented (most recent: July 2026), supplemented by automated scanning on every change and automated review on every pull request. Independent penetration testing has not yet been performed and will be commissioned as the user base and team grow, per the parent policy. Lessons from incidents and drills feed back into this policy, the drill catalogue, and the automated test suite.

9. Document Control

Version	Date	Change
1.0	2026-07-14	Initial adoption, elaborating Information Security Policy §5.